

agri_ai_core 소스코드 파일 트리

총 70개 실행 Python 파일 (__init__/_main_ 제외) | 2026-04-16

agri_ai_core/

- logs.py — 로그 모듈 — 공용 로거 생성 + 일별 로테이션 + 오래된 로그 자동 정리
- scheduler.py — 백그라운드 서비스 러너
- startup.py — 애플리케이션 초기화 모듈
- api/
 - app.py — FastAPI 애플리케이션: REST API 서버, LLM 질의, RAG 엔드포인트.
 - models.py — API 요청/응답 Pydantic 모델 정의.
 - rpi_router.py — 라즈베리파이 관리 API: SSH 재시작 및 센서 상태 모니터링.
 - voice_router.py — 음성 API 라우터: STT/TTS REST API 엔드포인트.
- config/
 - constants.py — 상수 모듈 - 시스템 전역 상수, 임계값, 스케줄링 설정, Ollama/모델 헬퍼.
 - mappers.py — 매핑 모듈 - 센서/릴레이 한글-영문 필드 매핑 및 house_id별 매핑 반환.
 - settings.py — 설정 모듈 - 환경변수 기반 애플리케이션 설정 로드 (dataclass + lru_cache ...)
- src/
 - ai/
 - conversation_store.py — 대화 히스토리 저장소
 - farm_cache.py — 농장 이름 캐시 — farm_id → farm_name 매핑 (프로세스 수명)
 - file_processor.py — 파일 처리 모듈 — 업로드 파일(CSV/Excel/PDF/텍스트)을 LLM 입력 형식으로 변환...
 - llm_client.py — LLM 클라이언트 핵심 모듈 — Ollama API 통신 및 Tool Use 응답 생성.
 - llm_message_utils.py — LLM 메시지/응답 유틸 — pure helper 함수 모음
 - llm_response.py — LLM 응답 후처리 모듈 — 도구 결과 정제, 응답 필터링, 마크다운 표 정렬.
 - llm_response_processing.py — LLM 응답 후처리 — 질문 분류, 응답 유형 판정, 재시도 검증, 대화 컨텍스트 구성.
 - llm_transport.py — LLM 전송 계층 — Ollama 모델 선택, GPU 비율 산출, 3가지 전송 방식.
 - llm_warmup.py — LLM 워밍업 — 백그라운드에서 첫 LLM 호출로 모델 VRAM 적재 + 추론 준비.
 - mcp_client.py — MCP (Model Context Protocol) 클라이언트 — MCP 서버(web-sear...
 - mcp_utils.py — MCP 순수 유틸리티 — 상태/통신 없는 pure helper 함수 모음
 - opinet_tools.py — Opinet 유가정보 도구 — 전국/시도/시군구 평균 유가, 최저가 조회
 - query_handler_simple.py — LLM 쿼리 핸들러 — Tool Use 방식으로 사용자 질문 처리 및 SSE 스트리밍.
 - query_utils.py — 질의 처리 공용 유틸 — query_handler_simple.py에서 분리된 pure hel...
 - stats_collector.py — 질의 통계 수집기 — LLM 응답 시간, 도구 호출, 검색 성공률 인메모리 수집.
 - tools_definition.py — LLM Tool Use 도구 정의
 - tools_executor.py — LLM Tool 실행기 — LLM이 요청한 도구(검색, DB, 릴레이 등)를 실행.
 - tools_search.py — LLM Tool — 웹 검색 및 URL 본문 가져오기 모듈
 - tools_utils.py — 도구 실행 공용 유틸 — tools_executor.py에서 분리된 pure helper
 - utils.py — AI 공통 유틸리티 — Think-tag 제거, 한국어 감지 등 공유 기능.
 - learning/
 - data_analyzer.py — 데이터 분석 — 센서/릴레이 이력 통계 분석 + 최적 환경 조건 도출
 - growth_rag_processor.py — 생육 기반 인과 관계 RAG: 환경 통계와 생육 데이터를 VectorDB에 저장.
 - model_trainer.py — 모델 학습 관리: 농장 데이터 분석 및 ChromaDB 저장.
 - pipeline/
 - answer_generator.py — 3단계 답변 작성: 수집된 데이터 기반 LLM 답변 생성 및 출처 선별.
 - data_collector.py — 2단계 데이터 수집: 도구 실행 및 LLM 기반 충분성 판단.
 - prompts.py — 3단계 파이프라인 전용 프롬프트 (분석 → 검증 → 답변).
 - question_analyzer.py — 1단계 질문유형분석: LLM 기반 질문 분석 및 데이터 수집 계획 생성.
 - validators.py — 2단계 데이터 검증: LLM 기반 수집 데이터 충분성 판단.
 - rag/
 - chunker.py — 텍스트 청킹: 문서를 의미 단위 청크로 분할하여 VectorDB에 저장.
 - constants.py — RAG 모듈 공통 상수 (하위 호출 re-export)
 - document_enricher.py — LLM 문서 요약 및 QA 쌍 생성: enriched 데이터를 VectorDB에 저장.
 - document_processor.py — 문서 처리 파이프라인: 파싱, 유형 감지, 청크 저장, LLM enrichment.
 - embedder.py — 텍스트 임베딩 생성: Ollama 기반 벡터 변환 및 캐시 관리.
 - reranker.py — LLM 기반 Reranker: 벡터 검색 후보를 관련성 점수로 재정렬.
 - chroma/

- |—— client.py — ChromaDB 클라이언트 계층
- |—— collections.py — ChromaDB 컬렉션 관리: 컬렉션 이름 반환 함수들.
- |—— config.py — ChromaDB 연결 설정 및 상수 정의.
- |—— loader.py — ChromaDB 데이터 로더: 미학습 데이터 조회 및 학습 상태 관리.
- |—— operations.py — ChromaDB 데이터 작업 계층
- |—— utils.py — ChromaDB 유틸리티: 메타데이터 변환, JSON 직렬화, 문서 ID 생성.
- |—— control/
 - |—— ai_control.py — AI 릴레이 제어 모듈.
 - |—— control_common.py — 제어 모듈 공통 상수, 매핑, 유틸리티.
 - |—— manual_control.py — 환경제어 모듈.
 - |—— relay_manager.py — 릴레이 제어 관리자 모듈.
 - |—— schedule_control.py — 릴레이 스케줄 제어 모듈.
 - |—— task_scheduler.py — 작업 스케줄러 모듈 (APScheduler 기반)
- |—— lotto/
 - |—— lotto_analyzer.py — 로또 추천 vs 당첨 비교 분석기 (LLM 전용)
 - |—— lotto_collector.py — 로또 당첨번호 수집기 — 동행복권 API에서 최신 당첨번호를 수집하여 DB에 저장.
 - |—— lotto_recommender.py — 로또 추천 (LLM 전용) — "시간의 파동(Time Wave)" 통계 + LLM 의도 해석
- |—— opinet/
 - |—— opinet_collector.py — Opinet 유가정보 수집기: 무료 API 데이터 수집 → PostgreSQL 저장.
- |—— postgresql/
 - |—— connection.py — PostgreSQL 연결 관리: 커넥션 풀 싱글톤 및 세션 컨텍스트 매니저.
 - |—— queries.py — PostgreSQL SQL 쿼리 상수 정의.
 - |—— reader.py — PostgreSQL 데이터 조회 계층 (read-only)
- |—— utils/
 - |—— conversion.py — 변환 유틸리티 — 센서/릴레이 데이터 추출, 안전한 숫자 변환
 - |—— date_utils.py — 날짜 유틸리티 모듈 - 날짜 파싱 및 포맷 변환 함수.
 - |—— error_utils.py — 예외 처리 유틸 — 공통 try/except 로깅 패턴 표준화
 - |—— json_utils.py — JSON 유틸 — 예외 안전한 JSON 직렬화/역직렬화
 - |—— validators.py — 데이터 검증 모듈 - is_true, clean_sensor_value, parse_boole...
- |—— voice/
 - |—— stt_engine.py — STT(음성→텍스트) 엔진 모듈
 - |—— tts_engine.py — TTS(텍스트→음성) 엔진 모듈